

Gerador de Currículo: *a primeira versão*

Último projeto do módulo: um formulário onde você preenche seus dados, vê a **prévia** do currículo ao lado e baixa um **PDF** pronto. O agente monta o formulário, a prévia em Markdown e o PDF com a biblioteca **fpdf2**, e você aprende a lidar com um erro clássico de layout de PDF e com caracteres especiais.

Formulário em duas colunas

Prévia em Markdown

fpdf2

st.download_button

Validação de e-mail

Acentos e emojis no PDF

Formação Claude Code 2026 · Baseado em py-pdf.github.io/fpdf2 (Text, Unicode), docs.streamlit.io (st.columns, st.download_button) e cursor.com/docs (Agent, Plan Mode, Browser), consultados em outubro de 2026 (fpdf2 2.8, Streamlit 1.65) · Código de referência testado em [projeto/](#) · Telas: print real

Um gerador, *duas aulas*

AULA 19 • HOJE

Formulário, prévia e PDF



AULA 20

Salvar, abrir, editar e excluir currículos num banco

ESCOPO DA V1

- ✓ Campos: nome, e-mail, telefone, cidade, resumo, experiências, formação, habilidades
- ✓ Experiências e formação: **uma por linha**; habilidades separadas por vírgula
- ✓ Prévia ao lado, atualizada enquanto você digita
- ✓ Botão para baixar o **PDF**
- ✓ Avisos: nome obrigatório, e-mail com formato estranho

DUAS SAÍDAS, UM SÓ DADO

Os dados do formulário viram um **dicionário**. A partir dele, `curriculo.py` gera duas coisas: o **Markdown** da prévia e o **PDF** do download. A tela não sabe montar PDF; o módulo não sabe nada de Streamlit.

```
pip install streamlit fpdf2
```

O pedido *da versão 1*

Crie um gerador de currículo em Streamlit, com layout largo e duas colunas:

- esquerda: formulário (nome, e-mail, telefone, cidade, resumo, experiências e formação uma por linha, habilidades separadas por vírgula);
- direita: prévia do currículo e botão "Baixar PDF".

Em `curriculo.py`, `para_markdown(dados)` e `para_pdf(dados)` usando `fpdf2`, A4, com títulos de seção. Avise se faltar o nome ou se o e-mail parecer inválido; nesses casos, não mostre o botão. Nome do arquivo: `curriculo_nome.pdf`.

DECISÃO	POR QUÊ
"uma por linha" / "por vírgula"	Formato simples de digitar e de separar no código
Markdown e PDF do mesmo dicionário	Prévia e PDF nunca ficam diferentes
fpdf2	Biblioteca Python pura, sem programas externos
"não mostre o botão"	Evita um PDF sem nome

Se você já tem um modelo de currículo de que gosta, **anexe um print** (aula 04) e peça o PDF "neste estilo".

Do formulário *ao Markdown*

```
"""Monta o currículo em Markdown (prévia) e em PDF (download)."""
from fpdf import FPDF

SECOES = [("resumo", "Resumo"), ("experiencias", "Experiência"), ("formacao", "Formação")]

def linhas(texto):
    """Uma entrada por linha, sem linhas vazias."""
    return [l.strip() for l in texto.splitlines() if l.strip()]

def habilidades(texto):
    return [h.strip() for h in texto.split(",") if h.strip()]

def para_markdown(dados):
    partes = [f"# {dados['nome']}"]
    contato = " · ".join(v for v in (dados["email"], dados["telefone"], dados["cidade"]) if v)
    if contato:
        partes.append(contato)
    if dados["resumo"].strip():
        partes.append("## Resumo\n" + dados["resumo"].strip())
    for chave, titulo in SECOES[1:]:
        itens = linhas(dados[chave])
        if itens:
            partes.append(f"### {titulo}\n" + "\n".join(f"- {i}" for i in itens))
    lista = habilidades(dados["habilidades"])
    if lista:
        partes.append("## Habilidades\n" + "\n".join(lista))
    return "\n\n".join(partes)
```

- ✓ `linhas()` e `habilidades()` transformam texto livre em listas, ignorando linhas e itens vazios
- ✓ Seção vazia não aparece (sem "Formação" em branco)
- ✓ O contato junta só o que foi preenchido, separado por " · "
- ✓ `#` e `###` viram títulos na prévia; `-` vira lista

O `st.markdown` não executa HTML por padrão: um texto colado com tags aparece como texto, não vira código na página.

O PDF *com fpdf2*

```
def para_pdf(dados):
    pdf = FPDF(format="A4")
    pdf.set_margins(18, 18, 18)
    pdf.add_page()

    pdf.set_font("Helvetica", "B", 22)
    pdf.cell(0, 11, _latin1(dados["nome"]), new_x="LMARGIN", new_y="NEXT")
    contato = " | ".join(v for v in (dados["email"], dados["telefone"], dados["cidade"]) if v)
    pdf.set_font("Helvetica", "", 10)
    pdf.set_text_color(90, 90, 90)
    pdf.cell(0, 6, _latin1(contato), new_x="LMARGIN", new_y="NEXT")
    pdf.set_text_color(0, 0, 0)

    def titulo(texto):
        pdf.ln(4)
        pdf.set_font("Helvetica", "B", 12)
        pdf.set_text_color(200, 90, 50)
        pdf.cell(0, 7, _latin1(texto.upper()), new_x="LMARGIN", new_y="NEXT")
        pdf.set_draw_color(220, 200, 190)
        pdf.line(pdf.l_margin, pdf.get_y(), pdf.w - pdf.r_margin, pdf.get_y())
        pdf.ln(2)
        pdf.set_text_color(0, 0, 0)
        pdf.set_font("Helvetica", "", 10.5)

    if dados["resumo"].strip():
        titulo("Resumo")
        pdf.multi_cell(0, 5.5, _latin1(dados["resumo"].strip()), new_x="LMARGIN", new_y="NEXT")
    for chave, nome in SECOES[1:]:
        itens = linhas(dados[chave])
        if itens:
            titulo(nome)
            for item in itens:
                pdf.multi_cell(0, 5.5, _latin1(f"- {item}"), new_x="LMARGIN", new_y="NEXT")
    lista = habilidades(dados["habilidades"])
    if lista:
        titulo("Habilidades")
        pdf.multi_cell(0, 5.5, _latin1(", ".join(lista)), new_x="LMARGIN", new_y="NEXT")

    return bytes(pdf.output())
```

- ✓ A4 com margens de 18 mm
- ✓ `cell` : uma linha; `multi_cell` : texto que quebra em várias
- ✓ `titulo()` desenha cada seção: maiúsculas, cor e uma linha embaixo
- ✓ `bytes(pdf.output())` devolve o PDF pronto para o botão

Repare no `new_x="LMARGIN", new_y="NEXT"` em todo `multi_cell` . Sem ele, o PDF quebra (próxima seção).

"Not enough *horizontal space*"

Na primeira versão do código de referência, gerar o PDF terminava assim:

```
fpdf.errors.FPDFException: Not enough horizontal  
space to render a single character
```

- 1 Depois de um `multi_cell`, o fpdf2 deixa o cursor à direita do texto, por padrão.
- 2 O próximo `multi_cell(0, ...)` usa "o resto da linha", que agora é zero.
- 3 Não cabe nem um caractere: exceção.

Correção: mandar o cursor para a margem esquerda da linha seguinte depois de cada bloco:

```
pdf.multi_cell(0, 5.5, texto,  
               new_x="LMARGIN", new_y="NEXT")
```

Ao gerar o PDF aparece este erro: [cole o erro completo]. Explique a causa no fpdf2 e corrija em todos os `multi_cell`, não só no primeiro.

"Em todos, não só no primeiro": o agente às vezes corrige só a linha do erro, e o próximo `multi_cell` quebra do mesmo jeito.

Acentos, travessões e emojis

```
def _latin1(texto):  
    # As fontes padrão do PDF só têm caracteres latinos (acentos do português incluídos).  
    return texto.replace(".", "-").replace("-", "-").replace("—", "-").encode("latin-1", "replace").decode("latin-1")
```

As fontes padrão do PDF (Helvetica, Times, Courier) só têm o alfabeto **Latin-1**. Acentos do português (á, ç, ã) estão lá; travessão, aspas curvas e emojis não.

ENTRADA	NO PDF
São Paulo, ação, você	Igual
2024-atual (travessão)	2024-atual
Emoji 🌈, aspas “curvas”	Viram <code>?</code> , sem erro

QUER TUDO, ATÉ EMOJI?
Use uma fonte **Unicode** (arquivo .ttf, como DejaVu ou Noto Sans) com `pdf.add_font(...)`. Aí o PDF aceita qualquer caractere que a fonte tenha. A fonte precisa ir junto com o projeto.

O código de referência foi testado com acentos, travessão, aspas curvas e emoji: o PDF sai sempre, sem exceção.

A versão 1 *funcionando*

Gerador de Currículo

Seus dados

Nome completo

Ana Souza

E-mail

ana@exemplo.com

Telefone

(11) 98765-4321

Cidade / estado

São Paulo - SP

Resumo profissional

Desenvolvedora Python com foco em dados e automação.

Experiência (uma por linha)

Empresa Alfa · Desenvolvedora Python · 2024-atual

Empresa Beta · Estagiária de dados · 2022-2023

Formação (uma por linha)

Fatec São Paulo · Análise e Desenvolvimento de Sistemas · 2023

Habilidades (separadas por vírgula)

Python, SQL, Streamlit, Git, Pandas

Prévia

Ana Souza

ana@exemplo.com | (11) 98765-4321 | São Paulo - SP

Resumo

Desenvolvedora Python com foco em dados e automação.

Experiência

• Empresa Alfa · Desenvolvedora Python · 2024-atual

• Empresa Beta · Estagiária de dados · 2022-2023

Formação

• Fatec São Paulo · Análise e Desenvolvimento de Sistemas · 2023

Habilidades

Python, SQL, Streamlit, Git, Pandas

Baixar PDF

Ana Souza

ana@exemplo.com | (11) 98765-4321 | São Paulo - SP

RESUMO

Desenvolvedora Python com foco em dados e automação. Gosto de transformar planilhas em sistemas.

EXPERIÊNCIA

- Empresa Alfa · Desenvolvedora Python · 2024-atual

- Empresa Beta · Estagiária de dados · 2022-2023

FORMAÇÃO

- Fatec São Paulo · Análise e Desenvolvimento de Sistemas · 2023

HABILIDADES

Python, SQL, Streamlit, Git, Pandas

App · print real do projeto/ , com dados fictícios

PDF gerado pelo para_pdf()

Roteiro *de teste*

TESTE	ESPERADO
Abrir vazio	"Preencha o nome."; sem botão
E-mail "ana@"	"O e-mail parece inválido."
Tudo preenchido	Prévia completa e botão Baixar PDF
Deixar Formação vazia	A seção some da prévia e do PDF
Linhas em branco nas experiências	São ignoradas
Acentos, travessão, emoji	PDF gerado, sem erro
Texto muito longo no resumo	O PDF quebra a linha e cria nova página se precisar

@Browser Abra o gerador, preencha com dados fictícios, confira a prévia, baixe o PDF e abra o arquivo. Os acentos estão certos? Alguma seção vazia apareceu? Depois apague o nome e veja se o botão some.

Use **dados fictícios** nos testes com o agente. Currículo real tem telefone, e-mail e endereço: dados pessoais que não precisam passar pelo chat.

Checklist *e commit*

- ✓ Formulário e prévia lado a lado
- ✓ PDF com todas as seções preenchidas e nenhuma vazia
- ✓ Avisos de nome e e-mail
- ✓ Acentos certos no PDF; caracteres especiais não quebram
- ✓ Lógica em `curriculo.py` ; tela em `app.py`
- ✓ `requirements.txt` : `streamlit` e `fpdf2`

```
git add .  
git commit -m "Gerador de Currículo: formulário, prévia e PDF"
```

Ideias para treinar: escolher a cor dos títulos, incluir foto, gerar também em `.docx` . Na próxima aula, o foco é outro: guardar os currículos num banco.

Teste o que *ficou*

1. Por que prévia e PDF saem do mesmo dicionário?

A) É mais rápido

B) Para nunca ficarem diferentes

C) O fpdf2 exige

D) Para salvar no banco

B. Uma fonte de dados, duas saídas.

2. Qual era a causa do "Not enough horizontal space"?

A) Fonte grande demais

B) Cursor parado à direita depois do multi_cell

C) Página pequena

D) Falta de memória

B. new_x="LMARGIN", new_y="NEXT" resolve.

3. O que acontece com um emoji no PDF com fonte padrão?

A) Aparece colorido

B) Vira "?" (sem erro, no código de referência)

C) O app trava

D) Vira imagem

B. Para emoji, use fonte Unicode.

4. Por que testar com dados fictícios?

A) São mais bonitos

B) Dados pessoais reais não precisam passar pelo chat

C) O fpdf2 não aceita reais

D) É mais rápido

B. Privacidade.

Do formulário *ao PDF*

O gerador junta formulário, **prévia** em Markdown e **PDF** com fpdf2, tudo a partir do mesmo dicionário. Corrigimos o erro de espaço do `multi_cell` e tratamos caracteres fora do Latin-1. Na última aula do módulo, os currículos vão para um **banco de dados**.

`curriculo.py`

`para_markdown` · `para_pdf`

`fpdf2`

`new_x` · `new_y`

`Latin-1` × `Unicode`

Próxima aula → 20 · Projeto Gerador Currículo · Salvando no Banco de Dados

Fontes: py-pdf.github.io/fpdf2 (Text, Line breaks, Unicode); docs.streamlit.io (st.columns, st.text_area, st.markdown, st.download_button); cursor.com/docs (Agent Overview, Prompting agents > Image input, Browser), consultados em outubro de 2026. Código de referência em `projeto/`, testado com fpdf2 2.8.9 e Streamlit 1.65; o erro da seção 05 aconteceu e foi corrigido nesse código.